

Master 2 Informatique
Data sciences, infrastructure cloud et sécurité

Machine Learning By Practice

Fabrice LAURI
Maître de conférences
à l'Université de Technologie de Belfort-Montbéliard

General Overview

- **Introduction**
- **Data Visualization**
- **Supervised Learning**
- **Unsupervised Learning**
- **Dimensionality Reduction**
- **Reinforcement Learning**
- **Conclusion**

CM #1

Reinforcement Learning

- Introduction
- Mathematical Framework
- Standard Finite MDP algorithms

Plan

- Rational Agents
- What is reinforcement learning?
- Action Selection Strategies
- Mathematical framework of RL problems
- Techniques solving RL problems with finite MDP
- Limits of previous RL techniques
- Generalisation and approximation of functions

Rational Agents

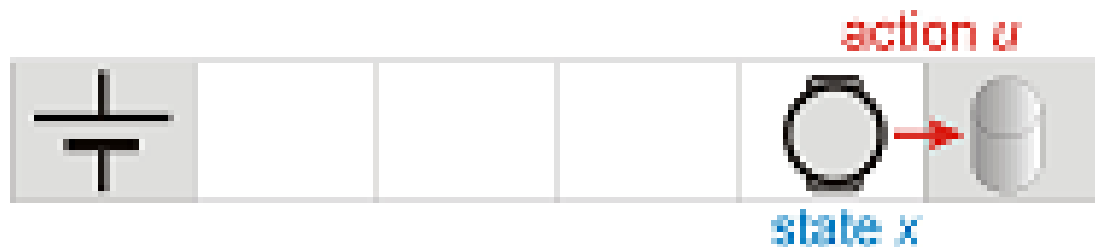
Reactive agents

Behavior of a reactive agent

A Reactive Agent behaves according to a function that maps action to each sequence of percepts, that is $\pi : S^* \rightarrow A$

Example

Hoover Agent...



Rational Behavior

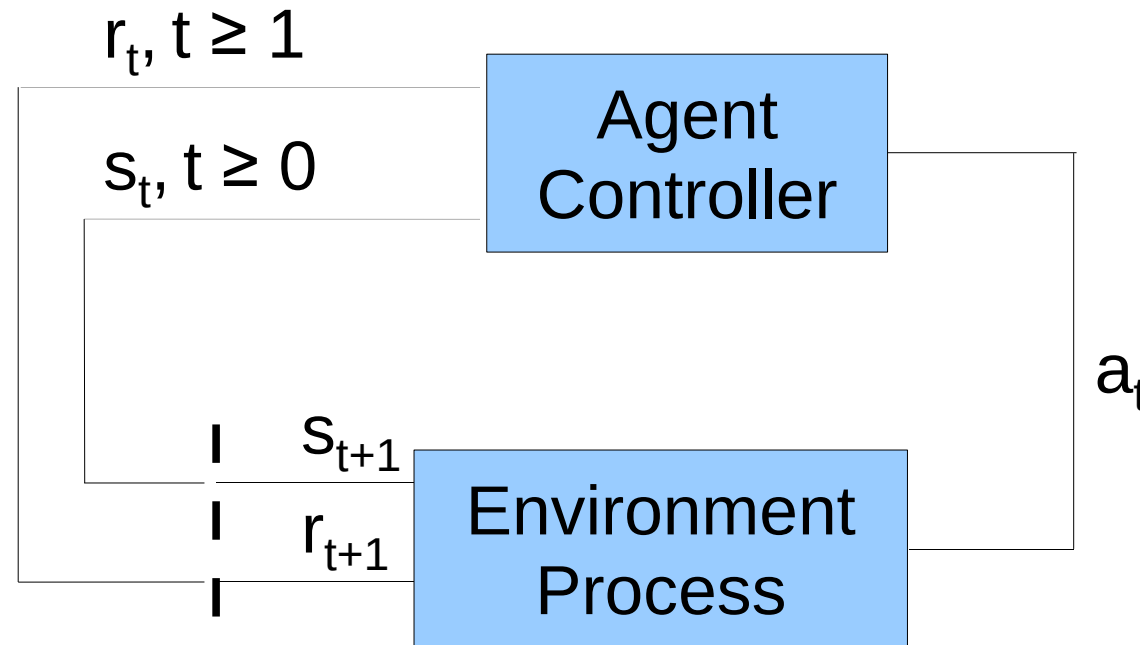
An agent is rational if it chooses, at every decision, an action that **maximises a given performance measure**, from the history of its perceptions and the knowledge about its environment.

This performance measure defines a success criteria upon the actions carried out by the agent.

Performance Measure : What the agent must do...

Pay attention : **rational $\neq$ omniscient !**

Sensori-motor Loop of the interactions of an agent with its environment

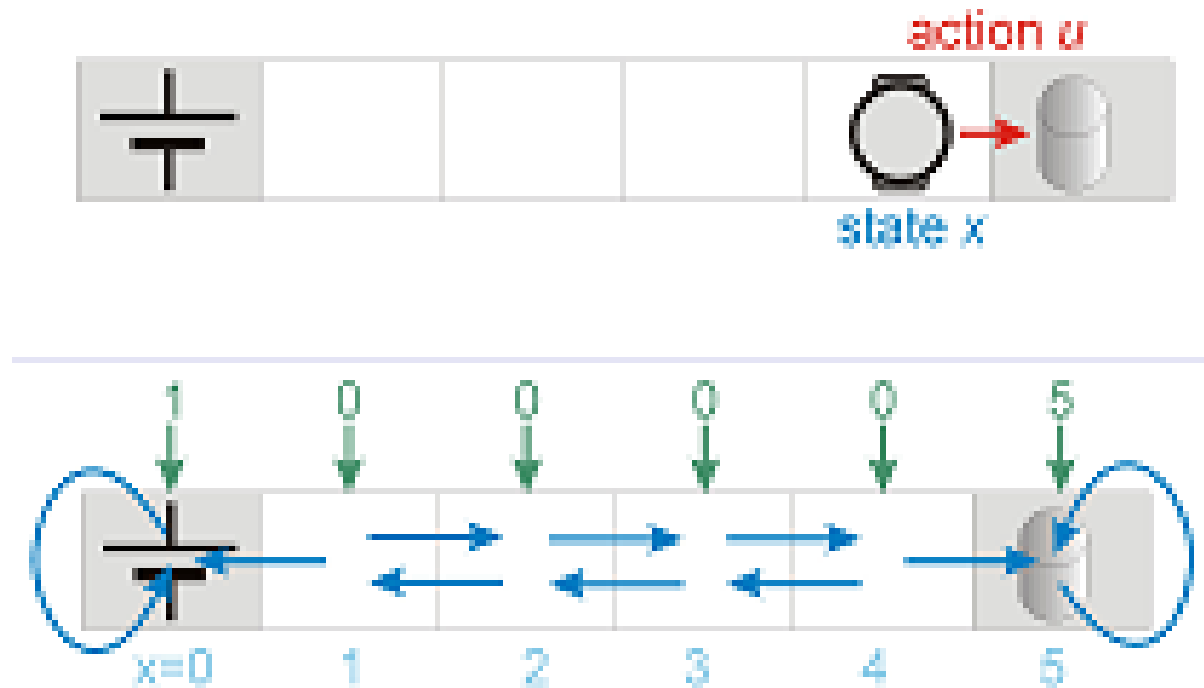


Interaction by the means of 3 types de signals:

- **State** signal
- **Action** signal
- **Reward** signal

Hoover Agent

Example :



Autonomy and Learning

According to the complexity of the environment, an agent can be autonomous:

- From its conception : agent architecture = set of rules of type condition-action
- While discovering its environment:
learning of new rules of type condition-action

The **features of the environment** influence the choice of the agent architecture...

Features of the environment

(cf. Russel & Norvig, 2010)

- Completely observable / partially observable
- Deterministic / stochastic
- Episodic Task / Continuous task
- Static / Dynamic
- State, Action and Time Spaces: discrete / continuous
- Environment with only one agent / multi-agent

Work Hypothesis

In this course, we assume the environment is:

- **Completely observable**
- Deterministic or stochastic
- Described like a episodic or continuous task
- **Static**
- Described by **finite** state, action and time spaces
- Is composed of **agents that cohabit** (agents are viewed as part of the environment from an agent viewpoint and they can not influence an agent while performing its task)
- **Some agents learn**

What is Reinforcement Learning?

What is Reinforcement Learning?

Learning by interacting with its environment for reaching a goal.

Examples : **Learn to**

- exit from a maze
- walk
- land with a helicopter
- control a set of elevators to reduce waiting time
- etc.

Learning by reinforcement = operative learning (*Skinner*)

What is Reinforcement Learning?

Learning by reinforcement = learning what to do for maximising the sum of rewards (reward=scalar number).

What to do = function that maps to each situation the best action to carry out.

Important features of RL:

- interactions via temporal, state, actions and reinforcement signals
- Search for the best actions by trials and errors
- dilemma exploitation/exploration
- delayed reward

Feature elements of a reinforcement learning agent

Two elements define the architecture of a learning agent:

- An action policy
- A value function

They are built from:

- A reward function R
- A state-transition function T (models the environment dynamic)

Feature elements of a reinforcement learning agent

Two elements define the architecture of a learning agent:

- **An action policy**
 - Determines the agent behavior
 - Mapping between encountered situation and action to realize
 - Stimulus-action rules (or condition-action)
 - Possibly stochastic

Feature elements of a reinforcement learning agent

Two elements define the architecture of a learning agent:

- A value function:
 - Maps every possible situation to the expected sum of rewards in the long-term if the agent starts from a given situation
 - represents the notion of long-term desirability of a situation
 - Must be estimated from the observation sequence the agent carries out during its entire life

Feature elements of a reinforcement learning agent

Built from:

- A reward function
 - Maps to each situation (or tuple state-action) a given reward
 - Represents the notion of immediate desirability (pleasure,pain) of a situation
 - Given directly by the environment when the agent has done an action
 - Goal of the agent: to maximise the sum of rewards
 - Can be stochastic

Feature elements of a reinforcement learning agent

Built from:

- A model of the environment dynamic
 - Allow the agent to know the next state(s) from the execution of an action in a state
 - If known by the agent:
 - Can predict the environnement dynamic
 - Can be used for action planification

The underlying model of a RL agent: Markov Decision Process

Un finite MDP is defined by:

- A finite set of states S (S includes the terminal states)
- A finite set of actions $A(s)$ for each state s
- A transition function between states

$$P_{ss'}^a = Pr(s_{t+1} = s' \mid s_t = s, a_t = a)$$

- A reward function

$$R_{ss'}^a = E\{r_{t+1} \mid s_t = s, a_t = a, s_{t+1} = s'\}$$

Environment
Dynamic

RL techniques are used when P and R are unknown.

Exploration-exploitation Tradeoff

In these RL techniques, at each time an agent has to face the dilemma between exploring a new action or exploiting the action that is known to be the best.



Exploring can be very dangerous...

One simple method to do exploration and exploitation and explore less and less during time: *Epsilon-Greedy*...

Action Selection Strategies

Action Selection Strategies

Which techniques can allow an agent to face the dilemma exploitation/exploration?

Let's study a simple case: a single situation, n possible actions.



Learning problem:

- Do 1000 times the choice among N possible actions.
- At each action a reward is given according to a static probability distribution.
- Maximizes the sum of rewards

Slot machine with n arms...

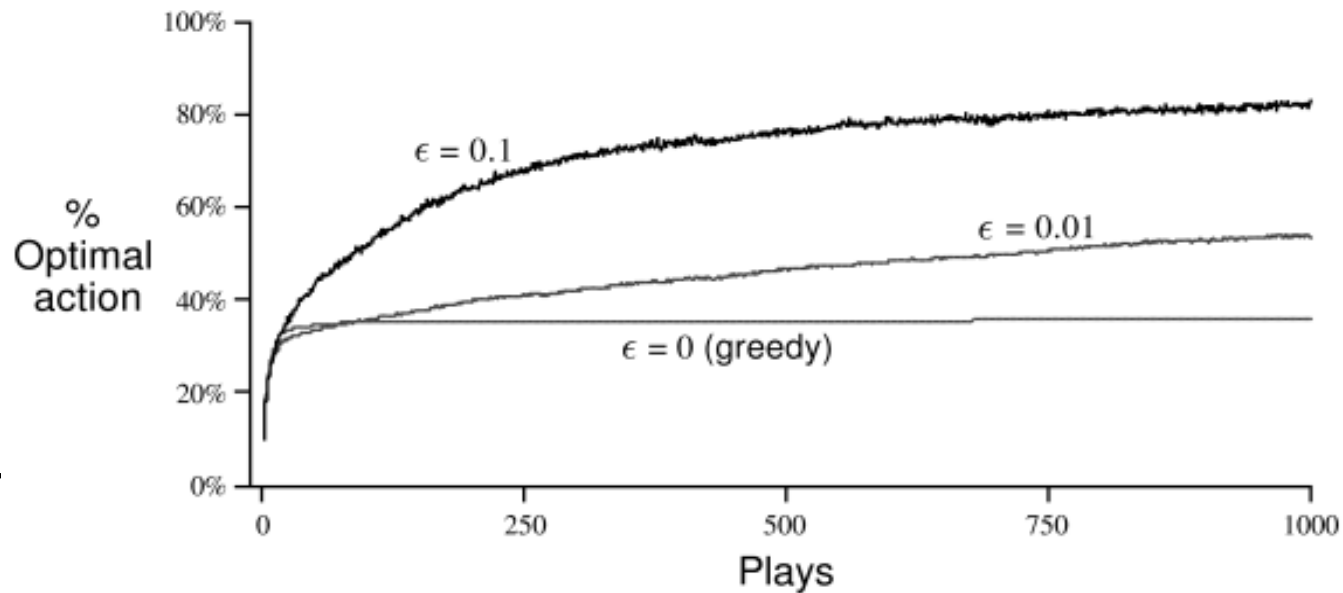
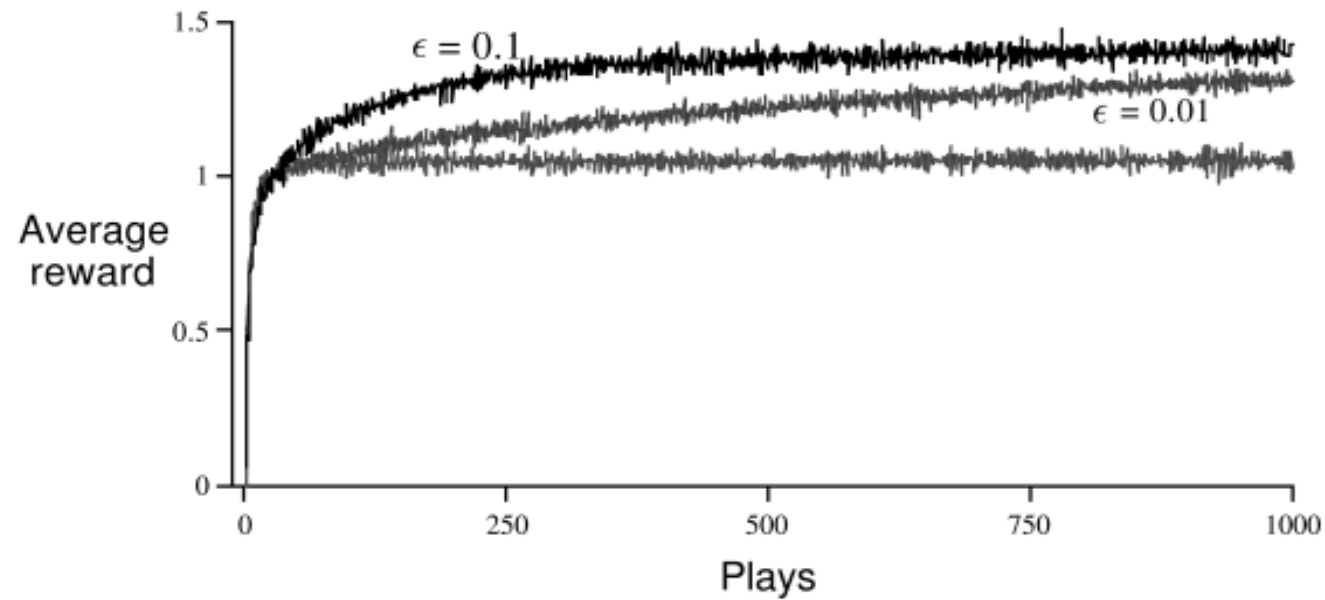
Action Selection Strategies

Problems:

- 1) How to estimate the value of an action?
- 2) How to choose the best action, on the basis of that value, and keeping the possibility of exploring new actions?

Answer...

Example with Epsilon-Greedy



Action Selection Strategies

Some (several) techniques can be used:

- Selecting actions with the technique **Greedy**

At each time, choose action that has the highest value.
Possible exploration if initial values are properly chosen.

- Selecting actions with the technique ***Epsilon-Greedy***

At each time, the best action is chosen with probability $1-\epsilon$, or one the possible action is chosen randomly and uniformly with a probability of ϵ .

R.L. Framework

Mathematical Framework of RL

Links between rewards and goals

At each instant t , the agent receives a reward $r_t \in \mathbb{R}$.

Its goal: to maximize the total amount of received rewards.

Some examples:

- robot learning to walk
- agent learning to exit from a maze
- robot learning to collect cans
- program learning to play chess

In all these examples :

Reward = what you want the agent to do

Mathematical Framework of RL

Cumulated Rewards

If the task is episodic: sequence of received rewards after time t :

$$R_t = r_{t+1} + r_{t+2} + \dots + r_T$$

T : final time of the interactions between the agent and the environment in an ***episode***.

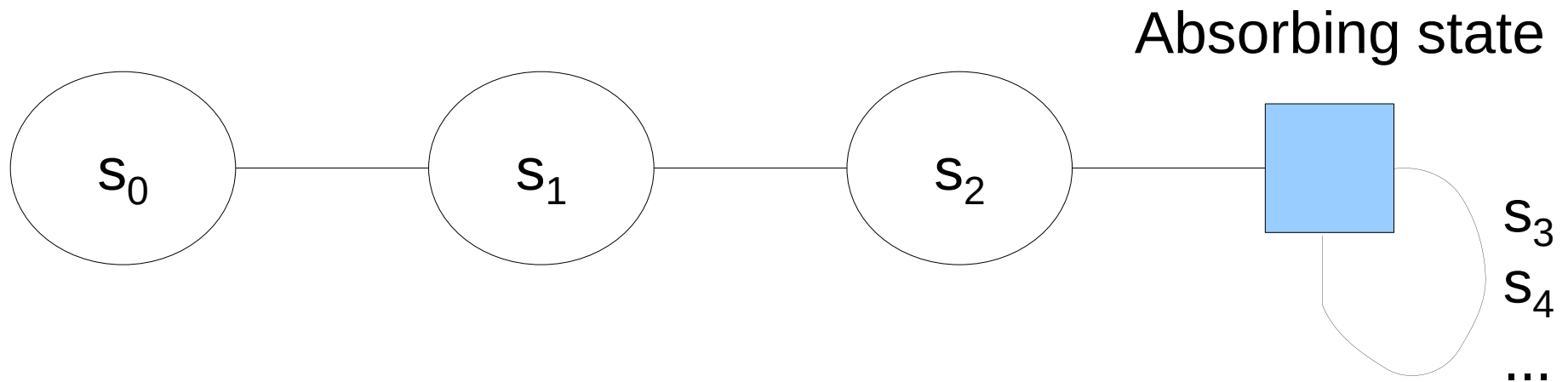
Otherwise the task is ***continuous***:

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

with: $0 \leq \gamma \leq 1$

Mathematical Framework of RL

Unified notation of episodic and continuous tasks



Sequence of received rewards after time t :

$$R_t = \sum_{k=0}^T \gamma^k r_{t+k+1}$$

avec soit : $T = \infty$ soit : $\gamma = 1$

Mathematical Framework of RL

Markov Properties on states

State = set of information known by the agent and that allow him to take a decision on the action to carry out.

Known information $\neq$ complete information

Dynamics of the environment :

$$Pr(s_{t+1} = s', r_{t+1} = r \mid s_t, a_t, r_t, s_{t-1}, a_{t-1}, r_{t-1}, \dots, r_1, s_0, a_0)$$

Mathematical Framework of RL

Markov Properties on states

A *markovian* state or a state that has the Markov property = state that stores the relevant non-ambiguous information about a situation, without describing the states-actions-rewards sequence that permit to reach this situation.

Examples and counter-examples :

- a position to tic-tac-toe
- a position to checkers
- a position to chess
- the position and the speed of an autonomous rocket

Dynamique d'un environnement markovien :

$$Pr(s_{t+1} = s', r_{t+1} = r \mid s_t, a_t)$$

Mathematical Framework of RL

Markov Decision Process

A RL task that satisfies the Markov property is a Markov Decision Process (MDP).

A finite MDP is defined by:

- A finite set of states S (S includes the terminal states)
- A finite set of actions $A(s)$ for each state s
- A transition function between states

$$P_{ss'}^a = Pr(s_{t+1} = s' \mid s_t = s, a_t = a)$$

- A reward function

$$R_{ss'}^a = E\{r_{t+1} \mid s_t = s, a_t = a, s_{t+1} = s'\}$$

Dynamic of the environment

Mathematical Framework of RL

Value functions

All efficient RL algorithms are based on a **value function**.

Value function:

- estimates the expected cumulated rewards from a given state (or from a state-action tuple)
- depends upon a given policy
- is defined by:

or :

$$V^{\pi}(s) = E_{\pi}\{R_t \mid s_t = s\} = E_{\pi}\left\{\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s\right\}$$

$$Q^{\pi}(s, a) = E_{\pi}\{R_t \mid s_t = s, a_t = a\} = E_{\pi}\left\{\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a\right\}$$

Mathematical Framework of RL

Bellman Equation on the value function

The value of a state s and the value of the next states of s are linked by the recurrent Bellman equation:

$$\begin{aligned} V^\pi(s) &= E_\pi\{R_t \mid s_t = s\} \\ &= E_\pi\left\{\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s\right\} \\ &= E_\pi\left\{r_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \mid s_t = s\right\} \\ &= \sum_{a \in A(s)} \pi(s, a) \sum_{s'} P_{ss'}^a \left[R_{ss'}^a + \gamma E_\pi\left\{\sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \mid s_{t+1} = s'\right\} \right] \\ &= \sum_{a \in A(s)} \pi(s, a) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')] \end{aligned}$$

Mathematical Framework of RL

Optimal value functions

Solving a RL task = finding the policy that maximises the value of every state.

The value functions can define a partial order on the policies, i.e.:

$$\pi \geq \pi' \Leftrightarrow V^\pi(s) \geq V^{\pi'}(s) \quad \forall s \in S$$

The optimal policies, denoted π^* , share the same value functions:

$$V^*(s) = \max_{\pi} \{V^\pi(s)\} \quad \forall s \in S$$

or:

$$\begin{aligned} Q^*(s, a) &= \max_{\pi} \{Q^\pi(s, a)\} \quad \forall s \in S \quad \forall a \in A(s) \\ &= E\{r_{t+1} + \gamma V^*(s_{t+1}) \mid s_t = s, a_t = a\} \end{aligned}$$

Mathematical Framework of RL

Bellman optimality equation on the value functions

Express the fact that the state value under the optimal policy must equal the cumulated rewards received when the best action is carried out from this state.

$$\begin{aligned} V^*(s) &= \max_{a \in A(s)} \{ Q^{\pi^*}(s, a) \} \\ &= \max_{a \in A(s)} \left\{ E_{\pi^*} \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right\} \right\} \\ &= \max_{a \in A(s)} \left\{ E_{\pi^*} \left\{ r_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \mid s_t = s, a_t = a \right\} \right\} \end{aligned}$$

Mathematical Framework of RL

Bellman optimality equation on the value functions

$$V^*(s) = \max_a \{ E_{\pi^*} \{ r_{t+1} + \gamma V^*(s_{t+1}) \mid s_t = s, a_t = a \} \}$$

$$= \max_{a \in A(s)} \left\{ \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^*(s')] \right\}$$

or:

$$Q^*(s, a) = E \{ r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a \}$$

$$= \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma \max_{a'} Q^*(s', a')] \quad (1)$$

Mathematical Framework of RL

Bellman optimality equation on the value functions

Solving the Bellman optimality equation for function V = solving a system of N equations with N unknowns, where N is the number of states.

When the dynamic of the environment is known:
Can be solved with any method that can solve systems of non-linear equations.

But the exact solving is generally impossible:

- The dynamic of the environment is not perfectly known
- Not enough computing or storage resources
- Markov property often unsatisfied

Example : backgammon...

Mathematical Framework of RL

Determining a policy from an optimal value function

Once V^* is determined for every state, how to define an action policy?

One solution: building the greedy policy (from V^* or Q^*) is an optimal policy.

Searching the best action a^* in every state s :

- From V^* : by realizing each possible action a from s and selecting the one that maximises $V^*(s')$ (where s' is the next state)

In this case the dynamic of the environment must be known...

- From Q^* : look at the action that maximises $Q^*(s,a)$

Resolution techniques of RL problems:

- Q-Iteration
- Policy Iteration
- Temporal-Difference Learning Techniques

Technique Q-Iteration

Technique Q-iteration

Input: functions P , R et *discount factor* γ

- 1: Initialize Q-function, e.g. $Q_0 \leftarrow 0$
- 2: **repeat** at each iteration $l = 0, 1, 2 \dots$
- 3: **for** every (s,a) **do**
- 4: Compute $Q_{l+1}(s,a)$ from $Q_l(s,a)$
- 5: **end for**
- 6: **until** $Q_{l+1} = Q_l$

By using equation
(1)

Output: $Q^* = Q_l$

Example

Q-iteration results for the deterministic cleaning-robot problem.

	$x = 0$	$x = 1$	$x = 2$	$x = 3$	$x = 4$	$x = 5$
Q_0	0; 0	0; 0	0; 0	0; 0	0; 0	0; 0
Q_1	0; 0	1; 0	0.5; 0	0.25; 0	0.125; 5	0; 0
Q_2	0; 0	1; 0.25	0.5; 0.125	0.25; 2.5	1.25; 5	0; 0
Q_3	0; 0	1; 0.25	0.5; 1.25	0.625; 2.5	1.25; 5	0; 0
Q_4	0; 0	1; 0.625	0.5; 1.25	0.625; 2.5	1.25; 5	0; 0
Q_5	0; 0	1; 0.625	0.5; 1.25	0.625; 2.5	1.25; 5	0; 0

h^*	*	-1	1	1	1	*
V^*	0	1	1.25	2.5	5	0

Technique Policy Iteration

Technique Policy iteration

Hypothesis:

- The environment dynamic is **known**.
- The tasks are **episodic**.

Features of the methods of this class:

- Learn from a state-action value function
- Policy evaluation and improvement cycles:

$$\pi_0 \xrightarrow{E} Q^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} Q^{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi^* \xrightarrow{E} Q^*$$

Policy iteration

1: Initialize policy h_0

2: **repeat** at every iteration $l = 0, 1, 2, \dots$

3: Find the Q-function of h_l : Q^{h_l}

◀ *Policy Evaluation*

4: $h_{l+1}(x) \in \operatorname{argmax}_u Q^{h_l}(x, u)$

◀ *Policy Improvement*

5: **until** $h_{l+1} = h_l$

Output: $h^* = h_l, Q^* = Q_l$

Example

TABLE 4.6

Policy iteration results for the deterministic cleaning-robot problem. Q-values are rounded to 3 decimal places.

	$x = 0$	$x = 1$	$x = 2$	$x = 3$	$x = 4$	$x = 5$
h_0	*	1	1	1	1	*
Q_0	0; 0	0; 0	0; 0	0; 0	0; 0	0; 0
Q_1	0; 0	1; 0	0; 0	0; 0	0; 5	0; 0
Q_2	0; 0	1; 0	0; 0	0; 2.5	1.25; 5	0; 0
Q_3	0; 0	1; 0	0; 1.25	0.625; 2.5	1.25; 5	0; 0
Q_4	0; 0	1; 0.625	0.313; 1.25	0.625; 2.5	1.25; 5	0; 0
Q_5	0; 0	1; 0.625	0.313; 1.25	0.625; 2.5	1.25; 5	0; 0
h_1	*	-1	1	1	1	*
Q_0	0; 0	0; 0	0; 0	0; 0	0; 0	0; 0
Q_1	0; 0	1; 0	0.5; 0	0; 0	0; 5	0; 0
Q_2	0; 0	1; 0	0.5; 0	0; 2.5	1.25; 5	0; 0
Q_3	0; 0	1; 0	0.5; 1.25	0.625; 2.5	1.25; 5	0; 0
Q_4	0; 0	1; 0.625	0.5; 1.25	0.625; 2.5	1.25; 5	0; 0
Q_5	0; 0	1; 0.625	0.5; 1.25	0.625; 2.5	1.25; 5	0; 0
h_2	*	-1	1	1	1	*

Temporal-Difference Learning Techniques

Temporal-Difference Learning techniques

Motivations :

- stop waiting the end of an episode to learn
- be able to learn a continuous task

Hypothesis:

The environment dynamic is **unknown**.

Features of the methods of this class:

- Learn from a state-action value function
- ***Bootstrapping***
(estimation from another learnt estimation)

Temporal-Difference Learning techniques

Evaluation with TD(0)

Rule of the update of the value of a tuple (s,a) at time t :

$$Q(s_t, a_t) \leftarrow (1 - \alpha) Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1})]$$

Target Value

Learning rate
to the target value

converges towards Q^π iff :

- α is constant and sufficiently small
- with a probability of 1 if α decreases according to the conditions of stochastic approximation

Temporal-Difference Learning techniques

A method based on TD : Q-learning

Initialize $Q(s, a)$ arbitrarily

Repeat (for each episode):

Initialize s

Repeat (for each step of episode):

Choose a from s using policy derived from Q (e.g., ε -greedy)

Take action a , observe r, s'

$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

$s \leftarrow s'$;

until s is terminal

Limits of the algorithms

Previous algorithms are :

- very sensitive to the number of states and actions (*state and action spaces complexities*).
- sensitive to the number of iterations used to learn the Q-function (*sample complexity*).

Questions:

- What if a 3D continuous position must be included with a state?
- What to do with continuous actions?

Techniques are practically limited to:

- 100,000 states, 10 actions and 5,000,000 iterations (offline)
- at most 100 states (for online learning)

Generalization and Function approximation

In previous techniques: Q-values are stored in a matrix.
→ represents **rote learning** (learning by heart)

When the state and action spaces are too big:

- storing these values in a matrix becomes impossible
- estimating these values requires too much data and so too much time to acquire them.

A solution → **generalizing**

Generalizing consists in estimating the Q-values of unseen pairs of (state,action) from Q-values met during learning.

Generalization and Function approximation

Several techniques to approximate the Q-function:

- Linear functions with:
 - State-dependent Basis Functions (BF) and discrete actions
 - State Aggregation
 - Tile coding
 - CMAC
 - Random *Radial Basis Functions* (RBF)
 - Fixed Sparse Representation
 - ...
- Deep Neural Networks → Deep RL
- ...

References

Reinforcement Learning

by Richard S. Sutton and Andrew G. Barto

1998

MIT Press, Cambridge MA

ISBN-10: 0-262-19398-1

ISBN-13: 978-0-262-19398-6

(342 pages)

Available at: <http://webdocs.cs.ualberta.ca/~sutton/book/ebook/the-book.html>

